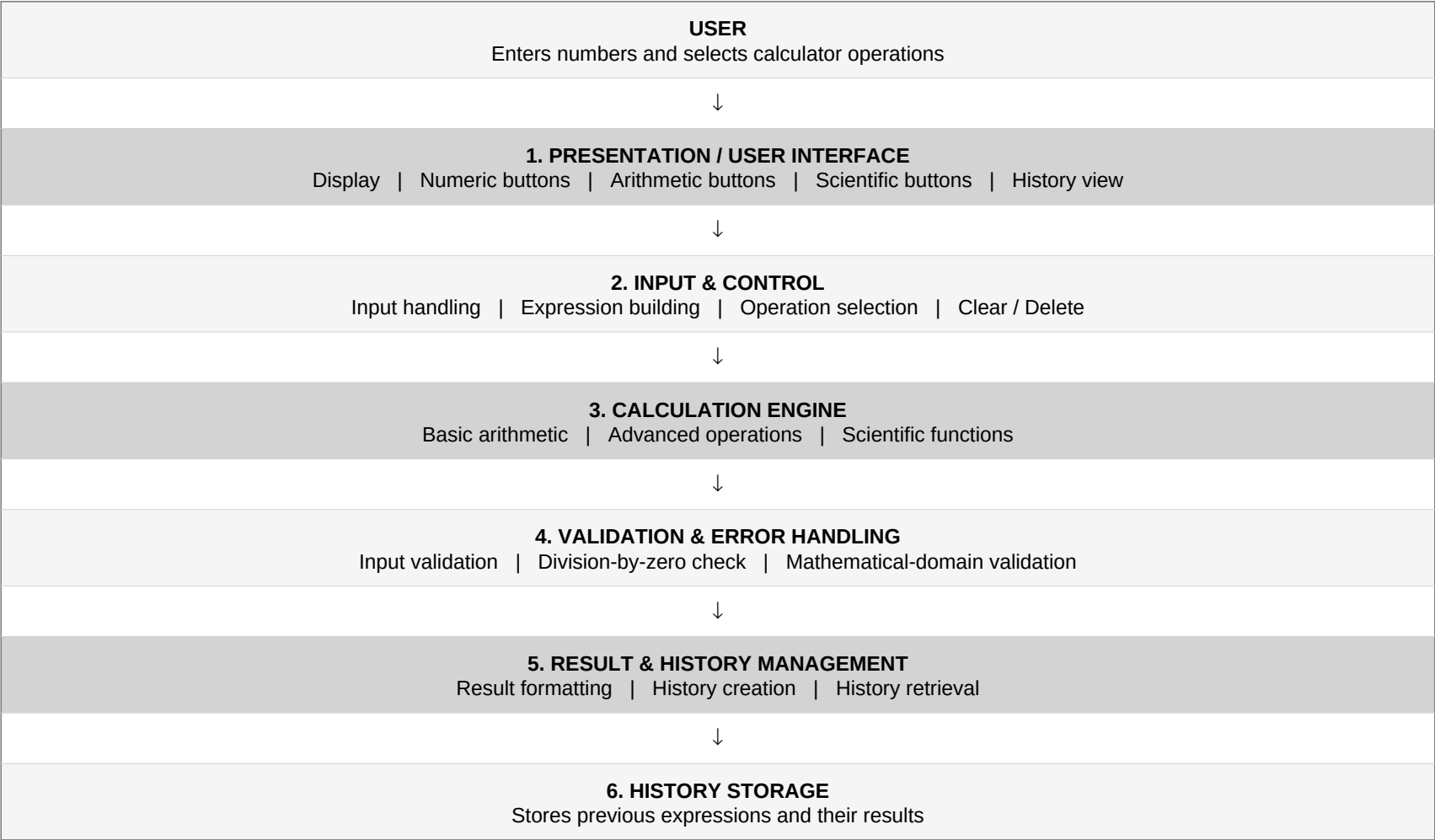


SYSTEM ARCHITECTURE

Smart Scientific Calculator | Incremental Software Development Model | SEPM

1. Architecture Overview

The Smart Scientific Calculator uses a modular layered architecture. Each layer has a specific responsibility, making the system easier to understand, test, maintain, and extend through successive increments.



2. Component Responsibilities

Component	Responsibility
Presentation / User Interface	Provides the calculator display, numeric controls, arithmetic controls, scientific controls, clear/delete controls, and access to history.
Input & Control	Accepts user input, builds the current expression, identifies the selected operation, and controls clear/delete actions.
Calculation Engine	Performs addition, subtraction, multiplication, division, modulus, percentage, square, power, square root, reciprocal, sine, cosine, tangent, logarithm, and natural logarithm.
Validation & Error Handling	Checks invalid input and mathematical conditions such as division by zero and unsupported mathematical domains.
Result & History Management	Formats calculation results for display and manages creation and retrieval of calculation-history entries.
History Storage	Stores completed expressions and their results for later viewing.

3. Data / Operation Flow

Step	System Flow
1	User enters numbers or selects a calculator operation.
2	User Interface sends the input to the Input & Control layer.
3	Input & Control interprets the requested operation.
4	Calculation Engine performs the mathematical operation.
5	Validation checks for invalid input or mathematical conditions.
6	Result is formatted and returned to the User Interface.
7	Completed calculation is sent to History Management when history is enabled.
8	User can retrieve previous calculations from History Storage.

4. Incremental Architecture Plan

Increment	Functionality Added
Increment 1	User Interface + Input/Control + basic Calculation Engine for addition and subtraction + clear + result display.
Increment 2	Add multiplication, division, modulus, percentage and delete; strengthen validation.
Increment 3	Add advanced operations: square, power, square root and reciprocal.
Increment 4	Add scientific functions: sine, cosine, tangent, logarithm and natural logarithm.
Increment 5	Add calculation history, final error-handling improvements, and UI refinement.

5. Architecture Principles

Principle	Application in the Project
Modularity	Calculator operations are organized into logical components so individual functions can be extended or modified.
Separation of concerns	Interface, input processing, calculation, validation, and history have separate responsibilities.
Incremental extensibility	New operations can be added in later increments without redesigning the complete system.
Testability	Calculation and validation logic can be tested independently from the interface.
Traceability	Architecture components can be mapped to SRS requirements and use cases.

6. Architecture-to-Requirements Traceability

Architecture Component	Related Requirements / Use Cases
User Interface	FR-16, FR-17, FR-20; UC-08 and all calculation use cases.
Input & Control	FR-01 to FR-18; UC-01 to UC-09.
Calculation Engine	FR-01 to FR-15; UC-01 to UC-07.
Validation & Error Handling	FR-19; UC-04, UC-06, UC-07 and UC-10.
Result & History Management	FR-18, FR-20; UC-09 and calculation use cases.
History Storage	FR-18; UC-09.

7. Design Note

This is the logical architecture for the project. The exact programming language, framework, and storage implementation can be selected during detailed design and implementation. The logical responsibilities shown here should remain consistent with the SRS and use cases.